

Seleção do k -ésimo menor elemento

Aula 8

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Introdução
- 2 Seleção com tempo esperado linear
- 3 Seleção com tempo de pior caso linear
- 4 Exercícios

Motivação

- ▶ considere o k -ésimo menor elemento de um conjunto de n elementos
- ▶ o **mínimo** de um conjunto de n elementos é o primeiro menor elemento do conjunto
- ▶ o **máximo** de um conjunto de n elementos é o n -ésimo (ou último) menor elemento do conjunto
- ▶ o **elemento médio** ou **mediana** de um conjunto de n elementos é o i -ésimo menor elemento desse conjunto, onde $i = \lfloor (n+1)/2 \rfloor$ (**mediana inferior**) e $i = \lceil (n+1)/2 \rceil$ (**mediana superior**)

Motivação

Problema da seleção do k -ésimo menor elemento:

Entrada: um conjunto A de n números e um número inteiro k , com $1 \leq k \leq n$

Saída: o elemento $x \in A$ que é maior que exatamente $k - 1$ elementos de A

- ▶ podemos resolver o problema da seleção em tempo $O(n \lg n)$, já que podemos ordenar os elementos usando o `HEAPSORT` ou o `MERGESORT` e então mostrar o k -ésimo elemento do vetor resultante
- ▶ veremos algoritmos mais rápidos nesta aula

Motivação

- ▶ é fácil encontrar o elemento mínimo? quantas comparações são realizadas? este é o melhor limitante? por quê?
- ▶ esse raciocínio é equivalente para o elemento máximo?
- ▶ e como fazemos para encontrar o elemento mínimo e o elemento máximo simultaneamente? quantas comparações são realizadas? será que conseguimos desenvolver um algoritmo que faz no máximo $3\lfloor n/2 \rfloor$ comparações?

Seleção com divisão-e-conquista

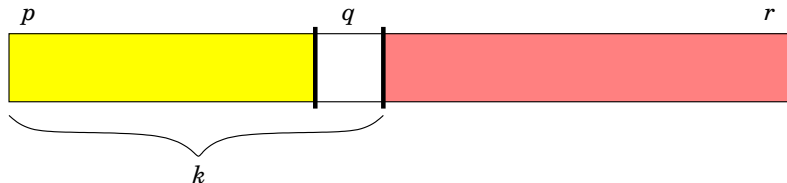
- ▶ o problema geral da seleção, isto é, o problema de encontrar o k -ésimo menor elemento de um conjunto com n elementos, parece mais difícil do que o problema de encontrar apenas o mínimo ou máximo
- ▶ surpreendentemente, o tempo de execução assintótico para os dois problemas é o mesmo: $\Theta(n)$
- ▶ apresentamos agora um algoritmo de divisão-e-conquista para o problema da seleção
- ▶ usa o RANDOMIZED-PARTITION do algoritmo QUICKSORT

Seleção com divisão-e-conquista

RANDOMIZED-SELECT(A, p, r, k)

```
01.  se  $p = r$ 
02.    devolva  $A[p]$ 
03.   $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$ 
04.   $i \leftarrow q - p + 1$ 
05.  se  $k = i$ 
06.    devolva  $A[q]$ 
07.  senão se  $k < i$ 
08.    devolva RANDOMIZED-SELECT( $A, p, q - 1, k$ )
09.  senão
10.    devolva RANDOMIZED-SELECT( $A, q + 1, r, k - i$ )
```

Seleção com divisão-e-conquista



Análise do algoritmo

- ▶ o tempo de execução de pior caso do algoritmo `RANDOMIZED-SELECT` é dado pela recorrência:

$$T(n) = T(n-1) + \Theta(n),$$

que é $T(n) = \Theta(n^2)$

- ▶ seu tempo de execução esperado é $\Theta(n)$

Análise do algoritmo

- ▶ seja $T(n)$ o tempo de execução esperado do algoritmo `RANDOMIZED-SELECT` quando executado sobre um vetor de entrada $A[p..r]$ com n elementos
- ▶ $T(n)$ é uma variável aleatória
- ▶ vamos obter um limitante superior para $E[T(n)]$
- ▶ o algoritmo `RANDOMIZED-PARTITION` devolve como pivô qualquer um dos elementos do vetor de entrada de forma igualmente provável

Análise do algoritmo

- ▶ para cada k tal que $1 \leq k \leq n$, o subvetor $A[p..q]$ tem k elementos, todos eles menores ou iguais ao pivô, com probabilidade $1/n$
- ▶ Para $k = 1, 2, \dots, n$, definimos variáveis aleatórias indicadoras X_k onde

$$X_k = I\{\text{o subvetor } A[p..q] \text{ tem exatamente } k \text{ elementos}\}$$

- ▶ assim, considerando que os elementos do vetor de entrada são distintos, temos

$$E[X_k] = 1/n$$

Análise do algoritmo

- ▶ quando executamos `RANDOMIZED-SELECT` e $A[q]$ é escolhido como pivô, não sabemos a priori se terminamos a execução imediatamente com a resposta correta, se uma chamada recursiva sobre o subvetor $A[p..q-1]$ é realizada ou se uma chamada recursiva sobre o subvetor $A[q+1..r]$ é realizada
- ▶ essa decisão depende de onde o i -ésimo menor elemento se encontra com relação a $A[q]$, com $i = q - p + 1$
- ▶ supondo que $T(n)$ é monotonicamente crescente, podemos limitar superiormente o tempo necessário para uma chamada recursiva pelo tempo necessário para uma chamada recursiva sobre a maior porção da entrada
- ▶ isto é, para obter um limitante superior, assumimos que o i -ésimo menor elemento está sempre no lado da partição com o maior número de elementos

Análise do algoritmo

- ▶ para uma dada chamada do RANDOMIZED-SELECT, a variável aleatória indicadora X_k tem valor 1 para exatamente um valor de k e 0 para todos os outros valores
- ▶ quando $X_k = 1$, os dois subvetores sobre os quais podemos realizar chamadas recursivas têm tamanhos $k - 1$ e $n - k$
- ▶ portanto, temos a seguinte recorrência

$$\begin{aligned} T(n) &\leq \sum_{k=1}^n X_k \cdot (T(\max\{k-1, n-k\}) + O(n)) \\ &= \sum_{k=1}^n X_k \cdot T(\max\{k-1, n-k\}) + O(n) \end{aligned}$$

Análise do algoritmo

- ▶ tomando os valores esperados, temos

$$\begin{aligned}
 E[T(n)] &\leq E\left[\sum_{k=1}^n X_k \cdot T(\max\{k-1, n-k\}) + O(n)\right] \\
 &= \sum_{k=1}^n E[X_k \cdot T(\max\{k-1, n-k\})] + O(n) \\
 &= \sum_{k=1}^n E[X_k] \cdot E[T(\max\{k-1, n-k\})] + O(n) \\
 &= \sum_{k=1}^n \frac{1}{n} \cdot E[T(\max\{k-1, n-k\})] + O(n)
 \end{aligned}$$

Análise do algoritmo

- ▶ considere a expressão $\max\{k-1, n-k\}$:

$$\max\{k-1, n-k\} = \begin{cases} k-1, & \text{se } k \geq \lceil n/2 \rceil, \\ n-k, & \text{se } k \leq \lceil n/2 \rceil. \end{cases}$$

- ▶ se n é par, cada termo de $T(\lceil n/2 \rceil)$ até $T(n-1)$ ocorre exatamente duas vezes no somatório e se n é ímpar, todos esses termos ocorrem duas vezes e $T(\lfloor n/2 \rfloor)$ ocorre uma vez
- ▶ logo,

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lceil n/2 \rceil}^{n-1} E[T(k)] + O(n)$$

Análise do algoritmo

- ▶ mostraremos agora que $E[T(n)] = O(n)$, usando o método da substituição
- ▶ suponha que $E[T(m)] \leq cm$ para todo $m < n$ e alguma constante c , que satisfaz as condições iniciais da recorrência
- ▶ assumamos que $T(n) = O(1)$ para n menor que alguma constante
- ▶ seja a uma constante tal que a função descrita em $O(n)$ é limitada por cima por an para todo $n > 0$

Análise do algoritmo

usando essa hipótese de indução, temos

$$\begin{aligned}
 E[T(n)] &\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an \\
 &= \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) + an \\
 &= \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1)\lfloor n/2 \rfloor}{2} \right) + an \\
 &\leq \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(n/2 - 2)(n/2 - 1)}{2} \right) + an
 \end{aligned}$$

Análise do algoritmo

(continuação)

$$\begin{aligned}
 E[T(n)] &\leq \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(n/2-2)(n/2-1)}{2} \right) + an \\
 &= \frac{2c}{n} \left(\frac{n^2-n}{2} - \frac{n^2/4-3n/2+2}{2} \right) + an \\
 &= \frac{c}{n} \left(\frac{3n^2}{4} + \frac{n}{2} - 2 \right) + an \\
 &= c \left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n} \right) + an \\
 &\leq \frac{3cn}{4} + \frac{c}{2} + an \\
 &= cn - \left(\frac{cn}{4} - \frac{c}{2} - an \right)
 \end{aligned}$$

Análise do algoritmo

- ▶ para completar a prova, devemos mostrar que $cn/4 - c/2 - an \geq 0$
- ▶ adicionando $c/2$ de ambos os lados, temos que $n(c/4 - a) \geq c/2$
- ▶ se escolhermos uma constante c tal que $c/4 - a > 0$, isto é, tal que $c > 4a$, podemos dividir ambos os lados por $c/4 - a$ e assim

$$n \geq \frac{c/2}{c/4 - a} = \frac{2c}{c - 4a}$$

- ▶ assim, se assumimos que $T(n) = O(1)$ para $n < 2c/(c - 4a)$, então $E[T(n)] = O(n)$
- ▶ portanto, podemos encontrar o k -ésimo menor elemento em tempo esperado linear, assumindo que os elementos são distintos

Considerações iniciais

- ▶ como o `RANDOMIZED-SELECT`, o algoritmo `SELECT` que veremos a seguir encontra o elemento desejado particionando recursivamente o vetor de entrada
- ▶ este algoritmo, no entanto, garante boas quebras durante o particionamento do vetor
- ▶ o algoritmo `SELECT` usa o algoritmo `PARTITION` do `QUICKSORT`, mas com uma modificação: o pivô é um parâmetro de entrada

Algoritmo SELECT

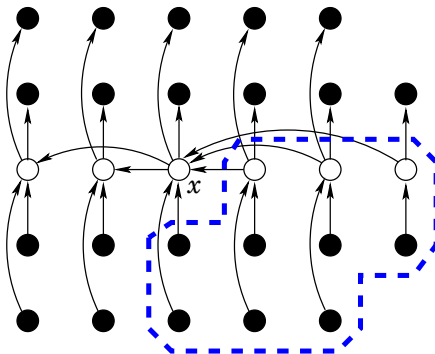
SELECT(A, k) // o vetor A tem n elementos

1. divida os n elementos de A em $\lfloor n/5 \rfloor$ grupos de 5 elementos cada, mais um último grupo com $n \bmod 5$ elementos
2. encontre a mediana dos $\lfloor n/5 \rfloor$ grupos, executando o INSERTIONSORT sobre cada grupo de ≤ 5 elementos e, em seguida, tomando a mediana do grupo
3. use SELECT recursivamente para encontrar a mediana x das $\lfloor n/5 \rfloor$ medianas encontradas no passo 2
4. particione A em torno da mediana-das-medianas x , usando a versão modificada do PARTITION; seja i tal que o número de elementos no lado esquerdo da partição é $i - 1$, x é o i -ésimo menor elemento e existem $n - i$ elementos no lado direito
5. se $k = i$, devolva x ; caso contrário, use SELECT recursivamente para encontrar o k -ésimo menor elemento no lado esquerdo da partição se $k < i$ ou o $(k - i)$ -ésimo menor elemento no lado direito da partição se $k > i$.

Análise do algoritmo `SELECT`

- ▶ quantos elementos são maiores que a mediana-das-medianas x após a execução do passo 2?

Análise do algoritmo SELECT



Análise do algoritmo SELECT

- ▶ quantos elementos são maiores que a mediana-das-medianas x após a execução do passo 2?
- ▶ pelo menos metade dos $\lceil n/5 \rceil$ grupos contribuem com pelo menos 3 elementos que são maiores que x , exceto o grupo contendo x e possivelmente o último grupo
- ▶ assim, o número de elementos maiores que x é pelo menos

$$3\left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2\right) \geq \frac{3n}{10} - 6.$$

- ▶ similarmente, pelo menos $3n/10 - 6$ elementos são menores que x
- ▶ assim, no pior caso, o passo 5 chama SELECT recursivamente sobre no máximo $7n/10 + 6$ elementos

Análise do algoritmo SELECT

- ▶ podemos agora estabelecer uma recorrência para o tempo de execução de pior caso $T(n)$ do algoritmo SELECT
- ▶ os passos 1, 2 e 4 têm tempo $O(n)$
- ▶ o passo 3 tem tempo $T(\lceil n/5 \rceil)$ e o passo 5 tem tempo $T(7n/10 + 6)$
- ▶ portanto, a recorrência para o tempo de execução de pior caso $T(n)$ do algoritmo SELECT é

$$T(n) = T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n)$$

Análise do algoritmo SELECT

- ▶ agora usamos o método da substituição para mostrar que $T(n) = O(n)$
- ▶ então,

$$\begin{aligned}
 T(n) &\leq T(\lceil n/5 \rceil) + T(7n/10 + 6) + an \\
 &\leq c(n/5 + 1) + c(7n/10 + 6) + an \\
 &= cn/5 + c7n/10 + 7c + an \\
 &= c9n/10 + an + 7c \\
 &= cn + (-cn/10 + an + 7c) \\
 &\leq cn,
 \end{aligned}$$

se $-cn/10 + an + 7c \leq 0$ que é equivalente a $c \geq 10a(n/(n-70))$, quando $n > 70$. Se supomos que $n \geq 140$, por exemplo, então $n/(n-70) \leq 2$ e, assim, escolhendo $c \geq 20a$ temos o resultado.

Exercícios

1. Escreva um algoritmo **MÍNIMO** que receba um vetor com n elementos distintos e devolva o elemento mínimo do vetor, usando $n - 1$ comparações. É possível fazer melhor que isso, isto é, devolver o elemento mínimo do vetor realizando menos que $n - 1$ comparações?
2. Escreva um algoritmo **MINMAX** que receba um vetor com n elementos distintos e devolva os elementos mínimo e máximo do vetor, usando no máximo $3\lfloor n/2 \rfloor$ comparações.
Sugestão: para cada par de elementos do vetor de entrada, compare esses elementos e depois compare o menor deles com o mínimo atual e o maior deles com o máximo atual, fazendo 3 comparações para cada 2 elementos de entrada.

Exercícios

- 9.2-1 Mostre que o algoritmo `RANDOMIZED-SELECT` nunca faz uma chamada recursiva a um vetor de tamanho 0.
- 9.2-4 Suponha que usamos o `RANDOMIZED-SELECT` para selecionar o elemento mínimo do vetor $A = \langle 3, 2, 9, 0, 7, 5, 4, 8, 6, 1 \rangle$. Descreva uma sequência de partições que resulta em um desempenho de pior caso do `RANDOMIZED-SELECT`.

Exercícios

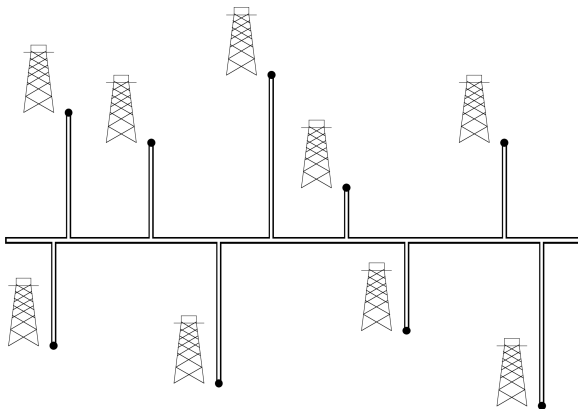
- 9.3-3 Mostre como o QUICKSORT pode ser modificado de forma a ter tempo de execução de pior caso $O(n \lg n)$, supondo que todos os elementos são distintos.
- 9.3-5 Suponha que você tenha uma subrotina “caixa-preta” para encontrar a mediana com tempo de execução de pior caso linear. Descreva um algoritmo simples, de tempo linear, que soluciona o problema da seleção, isto é, encontra o k -ésimo menor elemento.
- 9.3-8 Sejam $X[1..n]$ e $Y[1..n]$ dois vetores, cada um contendo n elementos arranjados em ordem crescente. Escreva um algoritmo com tempo de execução $O(\lg n)$ para encontrar a mediana dos $2n$ elementos nos vetores X e Y .

Exercícios

9.3-9 Professor Olay é um consultor de uma companhia de petróleo, que está planejando construir um grande oleoduto de leste a oeste em um campo com n poços de petróleo. A companhia quer conectar um oleoduto a partir de cada poço diretamente conectado ao oleoduto principal usando a menor rota possível ao norte ou ao sul, como mostra a figura a seguir. Dadas as coordenadas x e y dos poços, como o professor Olay deveria escolher a localização ótima do oleoduto principal, que minimizaria o comprimento total da rede? Mostre como determinar a localização ótima em tempo linear na quantidade de poços de petróleo.

Exercícios

9.3-9 (continuação)



Problemas

9-1 Maiores i elementos em ordem crescente

Dado um conjunto de n números, queremos encontrar os i maiores números e arranjá-los em ordem crescente usando um algoritmo baseado em comparações. Descreva o algoritmo que implementa cada um dos métodos a seguir com o melhor tempo de execução de pior caso e analise os tempos de execução dos algoritmos em termos de n e i .

- (a) Rearranje os números em ordem crescente e liste os i maiores.
- (b) Construa uma lista de max-prioridades a partir dos números e chame o algoritmo `EXTRAÍMÁXIMA` i vezes.
- (c) Use um algoritmo de seleção para encontrar o i -ésimo maior elemento, particione os números em torno desse pivô e ordene os i maiores números.